

Neural Style Transfer - An Evolutionary Analysis

1 Optimization based Neural Style Transfer

The first and foremost approach as described in the paper “A Neural Algorithm of Artistic Style” authored by Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge revolves around the concept of style-content separation. The paper claims that the style and content of an image, when processed by a Convolution Neural network (CNN) is separable.

1.1 Reconstructing Images from the Deeper Layers of a CNN

The paper hypothesizes that it is possible to reconstruct the contents and style of an image from the intermediate feature maps obtained when passed through a CNN. To prove this hypothesis, the authors use the following methodologies.

1.1.1 Content Reconstruction

1. A well trained model (VGG-19: Originally trained for image classification tasks on the ImageNet 1000 dataset) is taken for experimentation. The model has 19 layers and was modified by removing all Multi Layer Perceptron (MLP) heads for classification. Only the pure feature extracting CNN is isolated.
2. An image is taken into consideration and is passed onto the CNN. Consider a Feature map in a layer l to be K , a matrix of dim (say $n \times m$), and there are k of these feature maps (Each corresponding to a single kernel). The activations in these feature maps are flattened and laid side by side to obtain a P^l matrix of dim $(k, m \times n)$
3. The same process is repeated but with a white noise image and the corresponding P^l matrix is termed F^l and is noted.
4. In the forthcoming steps, the weights of the CNN is never modified, but the pixel values of the white noise image are the variables of interest, that we would like to tweak using backpropagation,
5. The loss function is defined as follows

$$L_c = \frac{1}{2} \sum_{i,j} (F_{ij}^l - P_{ij}^l)^2$$

Through backpropagation of this loss function, we reconstruct the content of the original image in the randomly initiated image. This process is repeated multiple times at different depths in the CNN, and the content quality depreciates as we go deeper.

1.1.2 Style Reconstruction

1. The same CNN model is chosen as mentioned in 1.1.1, without the classifier MLP. A styling image is taken into consideration.
2. A new matrix termed the Gram Matrix (G^l) is constructed from the F^l . All the elements in the F^l matrix (along the dim of number of kernels) is taken in order and arranged along rows and columns, and dot products are calculated to obtain G^l (Analogous to Attention scores in transformers with identical key/query vectors). It can be stated mathematically as,

$$G = \sum_k F_{ik}^l F_{jk}^l$$

Note that i, j correspond to the filter index and k corresponds to spatial index

3. The style loss, unlike content loss is not limited to a single layer, instead is calculated based on values from many inner layers. Scalar Style Loss function for a single layer is defined by,

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2$$

where A and G represent the gram matrices of the and the styling image and the white noise respectively. N_l is the number of kernels and M_l is the product of spatial dim.

4. The final style loss L_s is defined as the weighted sum of the individual E_l values.

$$L_s = \sum_{l=0}^L w_l E_l$$

Again by backpropagation and considering white noise pixel values to be the variables of interest, we extract the style representation

1.1.3 Style transfer

From section 1.1.1 and 1.1.2, we have the methods of style and content extraction/reconstruction techniques. The same processes are repeated but with two different pictures (input and the styling image) on the same white noise image. The net loss function is modified to account for both the content and style transfer by assigning weights to L_s & L_c . Therefore the final loss is given by,

$$L = \alpha L_c + \beta L_s$$

The lesser the value of $\frac{\alpha}{\beta}$ the higher is the priority given to styling over content transfer. Optimal values lie in the range of $(10^{-4}, 10^{-3})$. Also a cleaner version would simply be to divide the whole equation by β to prevent tweaking two hyperparameters.

1.2 Philosophy

Content is recorded/extracted in the feature maps, whereas style is extracted by calculating the correlation between the different layers. Style is obtained by using a multi-layer dependent loss function, but content extraction involves a mono-layered loss function. It is important to note that, the Gram matrix is not directly dependent on actual feature map activations (it is a derived quantity), nor does it involve the dimensions of the feature maps (They instead only take into account the number of kernels) as this dim is lost during the dot product calculation called Correlation.

Also, for the normalization factor in the Style loss function, because the dot product (to calculate G^l) scales linearly with M_l (The spatial dim), we divide by its square to account for the squared term normalization to eliminate spatial bias. The N_l term comes from the fact that the error involves the summation of N_l^2 terms.

2 Feed-Forward Style Transfer via AdaIN

This paper proposes a lightweight, feed forward mechanism that produces the combined advantageous outputs of all its predecessors. The proposed architecture evades the use of manual iterative optimization or redundant gram matrix calculations which is both computationally heavy and is time consuming.

2.1 Adaptive Instance Normalization (AdaIN)

The central doctrine of the paper follows from the great increase in performance when Instance normalization was introduced in existing predecessors. The core realization of this paper is the understanding of this shift in performance due to IN and exploiting it adaptively to control style transfer.

2.1.1 Instance Normalization

Instance normalization refers to the following process:

1. Say there are feature maps as a part of a tensor of dim (B, C, H, W).
2. Consider a single channel and a subsequent feature map (H, W)
3. We normalize this feature map with the mean and standard deviation calculated from this channel's corresponding feature map

2.1.2 Instance Normalization with Adaptive Affine parameters

AdaIN involves a subtle modification to the standard Instance Norm

1. IN is performed with learnable affine parameters
2. But in AdaIN, the parameters are not learned, instead they are obtained from the style image

3. The feature maps of the content image is instance normalized, and the transformation is inverted with the mean and standard deviation (affine parameters obtained from the style image's feature map)

$$\text{AdaIN}(f(c), f(s)) = \sigma(f(s)) \left(\frac{f(c) - \mu(f(c))}{\sigma(f(c))} \right) + \mu(f(s))$$

4. This effectively transfers style

2.2 Methodology

2.2.1 Training

1. A well trained model (VGG-19: Originally trained for image classification tasks on the ImageNet 1000 dataset) is modified by considering only the first 4 layers for experimentation. This is called the encoder.
2. Two images are taken into consideration, the content image ($\vec{c}$) and the style image ($\vec{s}$)
3. Both $\vec{c}$ and $\vec{s}$ are passed through the encoder. The feature maps obtained after the fourth ReLU layer of $\vec{c}$ is passed through the AdaIN layer, with the affine parameters being derived from the corresponding feature maps of $\vec{s}$. The corrected set of feature maps (the output tensor) is termed $\vec{t}$
4. This corrected feature map is passed through the decoder, which contains learnable/trainable parameters as opposed to the encoder which has fixed pre-trained weights. The decoder upsamples $\vec{t}$ by channel reduction and spatial increment to build the final output image.
5. $\vec{t}$ is considered the ground truth for loss function calculation. Similar to iterative optimization, this method also involves a uni-layered content loss and a multi-layered style loss.
6. The content loss is given by,

$$L_c = \sum_i (q_i - t_i)^2$$

where q_i represents the feature maps obtained when the decoder output is passed onto the encoder.

7. The style loss is the MSE between the statistical parameters (mean, standard deviation) in the feature maps of all the four layers. Mathematically given by,

$$L_s = \sum_{i=1}^4 (\mu(q_i) - \mu(s_i))^2 + \sum_{i=1}^4 (\sigma(q_i) - \sigma(s_i))^2$$

8. It is important to note that in all these backpropagations, the target variables are the learnable parameters of the decoder

9. The net loss is the linear combination of both L_s & L_c given by,

$$L = L_c + \lambda L_s$$

where λ corresponds to style weight and is a hyperparameter

2.2.2 Border Artifact prevention

Border artifacts occur predominantly due to the deconvolution/transposed-convolution during the upsampling process in the decoder, which results in unnatural grid like patterns or other irregularities in the final picture. To prevent this, two methods are devised.

(a) Nearest Neighbor upsampling

- In traditional transposed convolution the original set of feature maps are injected with zero padding to increase the spatial dimension
- However, this results in checkerboard artifacts/irregularities that arise from the uneven overlap during reconstruction. Hence the authors adhere to nearest neighbor upsampling.
- In this process, instead of internal zero padding, every element of the feature map is replaced by a (2×2) with the same value, effectively addressing the problem of uneven overlap.

(b) Reflection Padding

- Usually, zero border padding is used to further increase the spatial dimension
- But this forces the network to perceive the border as a thick zero layer, with the actual content being encapsulated at the center, which affects quality of the final image
- To prevent this, reflection padding is adopted, wherein the feature map activations are mirrored across the boundary. This substantially mitigates irregular artifacts

2.2.3 Inference

The major improvement in this methodology compared to its predecessors is that, it is capable of accepting a run-time (inference) style weight parameter.

During inference, the style weight parameter can be modified so as to transform the intermediate output tensor by combining it with a multiple of the original non-normalized content image output tensor. This can be mathematically stated as,

$$t' = (1 - \alpha)f(c) + \alpha \text{AdaIN}(f(c), f(s))$$

2.3 Philosophy

Style is encoded in the statistics of the feature maps. When the values within a feature map are normalized, the style is effectively lost, whereas the content information (placement of various features in the image) still remains intact, in the normalized value. Extending this observation, the authors note that, stripping the statistics of the content image in a deep layer which is capable enough to capture the nuances of the style, and swapping it with the statistics of the style image along the same depth, effectively transfers style, while preserving the content.

3 Comparison

1. AdaIN (A) completely abolishes the concept of Gram matrices, which it deemed redundant and that the extraction is possible by simply manipulating feature map statistics,
2. Gatys et al (G) involves an iterative optimization algorithm, which is considerably slow, despite being able to handle infinitely many style types during inference. The values that are targets of backpropagation are the pixel values of the output image itself. Inference is multi-step and iterative.
3. (A) involves a feed forward mechanism, which is faster, and still is capable to handle infinitely many styles. Also, in this case, the targets of backpropagation are the weights of the decoder and not the final pixel value. Inference is a single step process.
4. (G) is computationally heavy as opposed to (A), which has a comparatively lesser computational overhead owing to the absence of gram matrix calculation. (A) involves simple affine operations (Removal and application of statistical transforms and other normal feed forward calculations)
5. Due to the absence of direct upsampling layers (decoder) in (G), the final image is usually crisp in contrast to (A), which is prone to artifacts and bleeds.
6. (G) still remains the best in terms of capturing finer styling details, and (A) underperforms (G) for intricate style transfers owing to the oversimplification (G involves calculating correlation)
7. There is no clear distinction between the training and the inference phases of (G). For every style/content image pair, the model has to perform backpropagation
8. In contrast, there is a stark difference between the training and inference phases of (A). The model back propagates loss during training and strictly does not during inference.